University Help Desk System — Roles, CRUD Operations & Validation Improvements

A simple reference for the project defense: what each role can Create/Read/Update/Delete, and what to say if asked "how would you improve this?"

Note: this covers the three **system roles** built into the app (Student, Staff, Admin) — enforced by Spring Security in `SecurityConfig.java` (`/student/**` -> `ROLE_STUDENT`, `/staff/**` -> `ROLE_STAFF`, `/admin/**` -> `ROLE_ADMIN`). If you actually meant each teammate's individual contribution area on the group project, let me know and this can be redone from that angle instead.

1. How Roles Work in This System

Every user has exactly one `Role` (STUDENT, STAFF, or ADMIN), stored on the `users` table. When someone logs in, Spring Security reads that role and only lets them reach URLs matching it:

URL prefix	Who can access it
`/student/**`	STUDENT only
`/staff/**`	STAFF only
`/admin/**`	ADMIN only
`/reports`	STAFF or ADMIN
`/profile`, `/`	Any logged-in user

This check happens **before** any of your controller code runs, in `SecurityConfig.java`. A student who types `/admin/config` into their browser gets blocked automatically — no code in `AdminConfigController` has to check the role itself.

2. STUDENT — What They Can Do

Feature	Create	Read	Update	Delete
Account	Self-register (`POST /register`)	View own profile (`GET /profile`)	Edit own full name (`POST /profile`)	Not allowed (no self-delete)
Support Tickets	Submit a new ticket (`POST /student/tickets`)	View own ticket list and detail (`GET /student/tickets`, `GET /student/tickets/{id}`)	Reopen a closed ticket (`POST /student/tickets/{id}/reopen`)	Not allowed — tickets are never deleted, only resolved/closed, so there's a full history
Feedback	Rate a resolved/closed ticket, once only (`POST /student/tickets/{id}/feedback`)	View own submitted feedback (shown on the ticket page)	Not allowed	Not allowed

Feature	Create	Read	Update	Delete
Knowledge Base	—	Search articles, view an article, chat with the bot (`GET /student/kb`, `GET /student/kb/{id}`, `POST /student/kb/chatbot`)	—	—
Lost & Found	Report a lost/found item; submit a claim on an item (`POST /student/lost-found/report`, `POST /student/lost-found/{id}/claim`)	Browse/search listings, view item detail	Not allowed	Not allowed
Course Registration	Register for courses (`POST /student/registration`)	View own registration and open semesters	Change course selections by re-submitting the same form (acts as an update)	No explicit "cancel/drop registration" button exists yet

Built-in safety check: a student cannot view or act on *another* student's ticket even by guessing the URL/ID — `TicketController` checks `ticket.getStudent().getId().equals(principal.getUser().getId())` and blocks it otherwise. Good example to mention if asked about security.

3. STAFF — What They Can Do

Feature	Create	Read	Update	Delete
Support Tickets	Add a comment/resolution note (`POST /staff/tickets/{id}/comment`)	View their ticket queue and any ticket's detail (`GET /staff/tickets`, `GET /staff/tickets/{id}`)	Assign a ticket to self, change its status, escalate it (`POST /staff/tickets/{id}/assign`, `/status`, `/escalate`)	Not allowed
Knowledge Base	Create a new article, create a new category (`POST /staff/kb`, `POST /staff/kb/categories`)	View all articles, including drafts (`GET /staff/kb`)	Edit an existing article (same `POST /staff/kb`, with the article's id set)	Delete an article (`POST /staff/kb/{id}/delete`)
Reports	—	View ticket/feedback statistics (`GET /reports`, shared with Admin)	—	—
Dashboard	—	View own queue size, unassigned count, ticket count (`GET /staff/dashboard`)	—	—

Worth flagging as a gap: unlike the student side, the staff ticket endpoints don't check that a ticket actually belongs to that staff member's pool — any logged-in staff account can open or act on *any* ticket ID. See Section 5 (B1) for the fix.

4. ADMIN — What They Can Do

Feature	Create	Read	Update	Delete
Staff/Admin Accounts	Create a Staff or Admin account with an auto-generated temp password (`POST /admin/users`)	View all Staff/Admin accounts (`GET /admin/users`)	Activate/deactivate an account (`POST /admin/users/{id}/activate`, `/deactivate`)	Not allowed — accounts are disabled, never hard-deleted (keeps ticket/audit history intact)
Ticket Categories & Routing	Create a category, a staff work-pool, a routing rule; assign staff to a pool (`POST /admin/config/categories`, `/pools`, `/routing-rules`, `/pools/{id}/staff`)	View all config (`GET /admin/config`)	*(none for categories/pools themselves)*	Delete a routing rule; remove a staff member from a pool (`POST /admin/config/routing-rules/{id}/delete`, `/pools/{id}/staff/{id}/remove`)
Lost & Found Claims	—	View pending ownership claims (`GET /admin/lost-found`)	Approve or reject a claim (`POST /admin/lost-found/{id}/approve`, `/reject`)	—
Semester/Course Catalog	Create a semester, course, or course offering (`POST /admin/registration-config/semesters`, `/courses`, `/offerings`)	View the full catalog (`GET /admin/registration-config`)	*(none yet)*	*(none yet)*
Audit Log	— (written automatically by other actions)	View the system's audit trail (`GET /admin/audit-log`)	—	—
Reports/Dashboard	—	View system-wide stats (`GET /admin/dashboard`, `GET /reports`)	—	—

5. Validations to Improve the System

A. Validation already in place (good to mention — shows you understand what's there)

- Every form has server-side field validation (`@NotBlank`, `@Size`, `@Email`, `@NotNull`, `@PastOrPresent`) checked via `@Valid` + `BindingResult` before the service layer even runs.
- Password confirmation is checked (`AuthController.register()` compares `password` vs `confirmPassword` manually).
- Duplicate checks before saving: email, university ID, category name, pool name — each backed by a matching **unique constraint in the database** as a second line of defense, so even a bug in the Java check can't let a duplicate slip through.
- A student cannot view/reopen/give feedback on someone else's ticket (ownership check in `TicketController`).
- URL-level role restriction (Section 1) plus CSRF protection on every form (on by default with Spring Security + Thymeleaf).
- Real business rules enforced server-side: 18-credit registration cap, one claim per student per item, feedback only after a ticket is Resolved/Closed and only once, no two routing rules for the same category+priority.
- Passwords are BCrypt-hashed — never stored or logged as plain text.

B. Gaps found in the current code (concrete, fixable improvements)

1. **Staff can act on any ticket, not just their own queue.** `StaffTicketController` only checks `ROLE_STAFF` at the URL level — it never checks the ticket is in that staff member's pool or assigned to them. *Fix:* add the same kind of ownership check the student side already has, or use `@PreAuthorize` (the project already has `@EnableMethodSecurity` turned on in `SecurityConfig` but isn't using it yet).

2. **Notifications can never be marked as read.** The bell/count is displayed everywhere (`GlobalModelAttributesAdvice`), and a `markAllRead()` method already exists in `NotificationServiceImpl` — but no controller endpoint calls it. *Fix:* add a small `POST /notifications/mark-read`.

3. **Semesters, courses, and course offerings can't be edited or deleted** once created — only "create" endpoints exist. A typo'd course code or a semester that needs closing early has no UI fix. *Fix:* add update/close endpoints with the obvious guard (don't let a semester be edited after students have registered, for example).

4. **Ticket categories and staff pools can't be renamed or deleted**, only routing rules and pool memberships can. *Fix:* add rename/delete, blocking deletion of a category that still has open tickets pointing at it.

5. **Password policy is minimal (8 characters, nothing else)** — this is even flagged in the code's own comment as an intentionally open decision. *Fix:* add a pattern requiring at least one letter and one number (or stronger) for registration and any future "change password" feature.

6. **A couple of free-text fields have no maximum length** (`KbArticle.content`, `Feedback.comment`) while similar fields elsewhere do (`Ticket.description` is capped at 2000 characters). *Fix:* add a reasonable `@Size(max = ...)` to keep the database and page rendering predictable.

C. General hardening ideas worth mentioning (good practice, shows initiative)

- **File upload validation:** ticket attachments are capped at 10MB but nothing checks the file *type* — recommend allow-listing extensions (pdf, png, jpg, docx) and checking the real content type, not just the filename.
- **Login throttling:** no lockout after repeated failed logins yet, and no "forgot password" flow. Worth adding both for a production-ready system.
- **Status transition rules:** ticket status changes take a plain string; worth validating not just that it's a real status, but that the *transition* makes sense (e.g. block jumping straight from Closed to In Progress without going through

Reopened).

- **Rate-limit the chatbot endpoint** (`POST /student/kb/chatbot`) so it can't be spammed by a script.
- **Turn on** `@PreAuthorize` **at the service layer** as a second layer of defense underneath the URL-level rules already in `SecurityConfig` — the annotation is already enabled project-wide, just unused so far.
- **Protect reference data with dependents:** once category/pool editing is added, block deleting one that's still referenced by open tickets, rather than letting the database throw a raw foreign-key error.

6. Suggested Way to Present This to Your Lecturer

If asked "what would you improve," a strong answer walks through **one gap end-to-end**: pick #1 (staff acting on any ticket) or #5 (password policy), explain *why* it matters (security / data integrity), and describe the one-line fix in terms of the actual code (which file, which method) rather than a vague "we'd add more validation." That shows you understand the codebase, not just a checklist.